2024 CCF 非专业级别软件能力认证第一轮

（CSP-S1）提高级 C++语言试题

认证时间：2024 年 9 月 21 日 14:30~16:30

一、单项选择题(共 15 题，每题 2 分，共计 30 分;每题有且仅有一个正确选项)

1.在 Linux 系统中，如果你想显示当前工作目录的路径，应该使用哪个命令？（ ）

A.pwd

B.cd

C.ls

D.echo

【答案】A

【解析】

pwd （英文全拼：print work directory） 命令用于显示工作目录。

cd （英文全拼：change directory）命令用于改变当前工作目录的命令，切换到指定的路径。

ls （英文全拼： list directory contents）命令用于显示指定工作目录下之内容

echo 通常用于 shell 脚本中，用于显示消息或输出其他命令的结果。

2.假设一个长度为 n 的整数数组中每个元素互不相同，且这个数组是无序的。要找到这个数组中最大元素的时间复杂度是多少？（ ）

A.O(n)

B.O(log n)

C.O(n log n)

D.O(1)

【答案】A

【解析】 n 个元素找最大值，每个元素都要参与比较，要比较 n-1 次，时间复杂度为 O(n).

3.在 C++中，以下哪个函数调用会造成栈溢出？ （ ）

A. int foo() { return 0; }

B. int bar() { int x=1; return x; }

C. void baz() { int a[1000]; baz(); }

D. void qux() { return; }

【答案】C

【解析】 C 选项是递归函数，并且没有递归出口，会一直递归调用，直到栈溢出。

4.在一场比赛中，有 10 名选手参加，前三名将获得金银铜牌，若不允许并列，且每名选手只能获得一枚奖牌，则不同的颁奖方式共有多少种？ （ ）

A.120

B.720

C.504

D.1000

【答案】B

【解析】 依次选取金牌、银牌、铜牌选手，金牌 10 种选法，银牌就剩 9 种选法，铜牌剩余 8 种选法，就是 10 选 3 的全排列，A(10,3)=10*9*8=720.

5.下面哪个数据结构最适合实现先进先出（FIFO）的功能？ （ ）

A.栈

B.队列

C.线性表

D.二叉搜索树

【答案】B

【解析】实现先进先出(FIFO，First In First Out)功能的最适合数据结构是队列(Queue)

队列：先进先出。

6.已知 f(1) = 1,且对于 n>=2 有 f(n)=f(n-1)+f(□n/2□),则 f(4)的值为：（ ）

A.4

B.5

C.6

D.7

【答案】B

【解析】 递推计算即可。

f(1)=1

f(2)=f(1)+f(1)=2

f(3)=f(2)+f(1)=2+1=3

f(4)=f(3)+f(2)=5

7.假设一个包含 n 个顶点的无向图，且该图是欧拉图。以下关于该图的描述中哪一项不一定正确？（ ）

A.所有顶点的度数均为偶数

B.该图联通

C.该图存在一个欧拉回路

D.该图的边数是奇数

【答案】D

【解析】 欧拉无向图是指含有欧拉回路的无向连通图。欧拉回路，要求通过图中所有边，且每边仅通过一次，并且最终回到起点。

欧拉图中，奇点（度数为奇数的点）个数为 0，所有点度数均为偶数，A 正确。该图连通，所以 B 正确。存在欧拉回路，所以 C 正确。对于 D 选项，比如一个四个点

练成一个圈，四条边，也是一个欧拉图，显然边数不一定是奇数，所以 D 错误。

8.对数组进行二分查找的过程中，以下哪个条件必须满足？（ ）

A.数组必须是有序的

B.数组必须是无序的

C.数组长度必须是 2 的幂

D.数组中的元素必须是整数

【答案】A

【解析】 二分查找，序列必须是有序的。

9.考虑一个自然数 n 以及一个模数 m，你需要计算 n 的逆元（即 n 在模 m 意义下的乘法逆元)。下列哪种算法最为合适？（ ）

A.使用暴力方法依次尝试

B.使用扩展欧几里得解法

C.使用快速幂解法

D.使用线性筛法

【答案】B

【解析】计算逆元，可以使用扩展欧几里得解法。

10.在设计一个哈希表时，为了减少冲突，需要使用适当的哈希函数和冲突解决策略。已知某哈希表中有 n 个键值对，表的装载因子为 α（0<α<=1)。在使用开放地址法解决冲突的过程中，最坏情况下查找一个元素的时间复杂度为（ ）

A.O(1)

B.O(log n)

C.O (1/(1-α))

D.O(n)

【答案】D

【解析】

最好情况：当没有发生冲突时，直接找到目标元素，时间复杂度为 O(1)。

最坏情况：当所有元素都映射到同一个位置时，需要遍历整个哈希表，时间复杂度为 O(n).

平均情况：平均情况为 O(1/(1-a))，其中 a 为装载因子。

11.假设有一颗 h 层的完全二叉树，该树最多包含多少个节点（　）

A.2^h-1

B.2^(h+1)-1

C.2^h

D.2^(h+1)

【答案】A

【解析】 根结点深度为 1，h 层的完成二叉树最多有 2^h-1 个节点（满二叉树也是完全二叉树的一种形态）

12. 设有一个 10 个顶点的完全图，每两个顶点之间都有一条边，有多少个长度为 4 的环？（　）

A.120

B.210

C.630

D.5040

【答案】C

【解析】

首先，从 10 个顶点中选 4 个点的全排列为 A(10,4)，

其次，因为是一个环，相当于 4 个点的圆排列，所以要除以点数 4，

再者，因为一个环，顺时针看和逆时针看是对称的一样的，所以再除以 2，

综上，结果为 A(10,4)/4/2=10*9*8*7/4/2=630

13.对于一个整数 n，定义 f(n)为 n 的各位数字之和，问使 f(f(x))=10 的最小自然数 x 是多少？（　）

A.29

B.199

C.299

D.399

【答案】B

【解析】 将选项代入公式，f(f(199))=f(19)=10，所以选择 B 选项。

14.设有一个长度为 n 的 01 字符串，其中有 k 个 1，每次操作可以交换相邻两个字符。在最坏的情况下将这 k 个 1 移到字符串最右边所需要的交换次数是多少？（　）

A.K

B.K*(k-1)/2

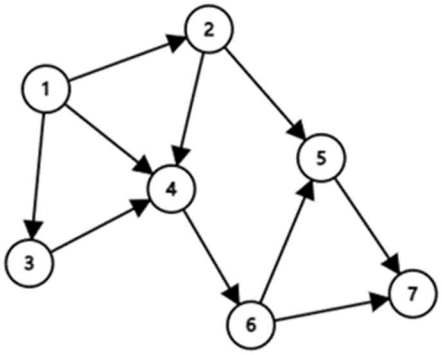
C.(n-k)*k

D.(2n-k-1)*k/2

【答案】C

【解析】 最坏情况是 k 个 1 全部在左侧，通过交换全部移动到最右侧。一个 1 移动到右侧交换(n-k)次，共需要(n-k)*k。

15.如图是一张包含 7 个顶点的有向图。如果要删除其中一些边，使得从节点 1 到节点 7 没有可行路径，且删除的边数最少，请问总共有多少种可行的删除边的集合？（　）



A.1　　B.2

C.3　　D.4

【答案】D

【解析】 可以删除{ (1,2)(4,6)} 或{ (2,5)(4,6)} 、或{(4,6)(5,7)}、或(6,7)(5,7) }，共 4 种情况。

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填√，错误填×，除特殊说明外，判断题 1.5 分，选择题 3 分，共计 40 分）

（1）

```
#include <iostream>
using namespace std;

const int N = 1000;
int c[N];

int logic(int x, int y) {
return (x & y) ^ ((x ^ y) | (~x & y));
}
void generate(int a, int b, int *c) {
for (int i = 0; i < b; i++) {
    c[i] = logic(a, i) % (b + 1);
}
```

```
14 }
15 void recursion(int depth, int *arr, int size) {
16 if (depth <= 0 || size <= 1)    return;
17 int pivot = arr[0];
18 int i = 0, j = size - 1;
19 while (i <= j) {
20     while (arr[i] < pivot)   i++;
21     while (arr[j] > pivot)   j--;
22     if (i <= j) {
23         int temp = arr[i];
24         arr[i] = arr[j];
25         arr[j] = temp;
26         i++;j--;
27     }
28 }
29 recursion(depth - 1, arr, j + 1);
30 recursion(depth - 1, arr + i, size - i);
31 }
32
33 int main() {
34     int a, b, d;
35     cin >> a >> b >> d;
36     generate(a, b, c);
37     recursion(d, c, b);
38     for (int i = 0; i < b; i++)   cout << c[i] << " ";
39     cout<<endl;
40 }
```

判断题

16. 当 1000>=d>=b 时，输出的序列是有序的（　　）

【答案】√

【解析】快排，d 是快排递归的层数，d>=b 时一定排好序

17. 当输入“5 5 1”时，输出为“1 1 5 5 5”（ ）

【答案】×

【解析】输入生成的序列为 5 5 1 1 5，d=1，模拟代码操作一次以后，序列变为 5 1 1 5 5

18.假设数组 c 长度无限制，该程序所实现的算法的时间复杂度是 O(b)的（ ）

【答案】×

【解析】快排的最坏时间复杂度时 O(n^2)

单选题

19.函数 int logic(int x,int y)的功能是（ ）

A.按位与

B.按位或

C.按位异或

D.以上都不是

【答案】B

【解析】可以发现都是位运算，因此考虑 x,y 分别为 0/1 的四种可能

x	y	logic(x,y)
0	0	0
0	1	1
1	0	1
1	1	1

20.（4 分）当输入为“10 100 100”时，输出的第 100 个数是（ ）

A.91

B.94

C.95

D.98

【答案】C

【解析】排序 100 层，最后一个数一定是生成的数里的最大值，相当于问(10|0)%101 到(10|99)%101 里的最大值,10|95 = 95 是能取到的最大值

(2)

```
1  #include <iostream>
2  #include <string>
```

```
3   using namespace std;
4
5   const int P = 998244353, N = 1e4 + 10, M = 20;
6   int n, m;
7   string s;
8   int dp[1 << M];
9
10  int solve() {
11  dp[0] = 1;
12  for (int i = 0; i < n; i++) {
13      for (int j = (1 << (m - 1)) - 1; j >= 0; j--) {
14          int k = (j << 1) | (s[i] - '0');
15          if (j != 0 || s[i] == '1')
16              dp[k] = (dp[k] + dp[j]) % P;
17      }
18      }
19  int ans = 0;
20  for (int i = 0; i < (1 << m); i++) {
21      ans = (ans + 1ll * i * dp[i]) % P;
22  }
23  return ans;
24  }
25  int solve2() {
26  int ans = 0;
27  for (int i = 0; i < (1 << n); i++) {
28      int cnt = 0;
29      int num = 0;
30      for (int j = 0; j < n; j++) {
31          if (i & (1 << j)) {
32              num = num * 2 + (s[j] - '0');
```

```
33                  cnt++;
34              }
35          }
36          if (cnt <= m)   (ans += num) %= P;
37      }
38      return ans;
39  }
40
41  int main() {
42          cin >> n >> m;
43          cin >> s;
44          if (n <= 20) {
45              cout << solve2() << endl;
46          }
47          cout << solve() << endl;
48          return 0;
49  }
```

假设输入的 s 是包含 n 个字符的 01 串，完成下面的判断题和单选题：

【代码大意】solve2()为统计 s 串所有长度不超过 m 的子序列 t 对应二进制值的和

solve()为统计 s 串所有长度不超过 m 的子序列 t(不能包含前导零)对应二进制值的和

判断题

21. 假设数组 dp 长度无限制，函数 solve( )所实现的算法时间复杂度是 O(n*2^m)

（ ）

【答案】√

【解析】状压 dp

22. 输入“11 2 10000000001”时，程序输出两个数 32 和 23（ ）

【答案】√

【解析】m=2，只有 0，1，01，10，11 共 5 种子序列，分别统计求和

23.（2 分）在 n<=10 时，solve( )的返回值始终小于 4^{10}（ ）

【答案】√

【解析】字符串为全 1 答案最大，此时值不超过 4^{10}。

单选题

24. 当 n=10 且 m=10 时，有多少种输入使得两行的结果完全一致？（　　）

A.1024

B.11

C.10

D.0

【答案】B

【解析】要结果一致就要保证所有包含前导零的方案的和为 0，也就是 s 只能是一段 1 加上一段 0，共有 11 中方案

25.当 n<=6 时，solve()的最大可能返回值为（　　）

A.65

B.211

C.665

D.2059

【答案】C

【解析】不难发现全 1 的时候答案最大

26.若 n=8，m=8，solve 和 solve2 的返回值的最大可能的差值为（　　）

A.1477

B.1995

C.2059

D.2187

【答案】C

【解析】01111111 是差值最大的方案，这种情况下，包含前导零的方案正好对应一个不包含的方案

(3)

```
1  #include <iostream>
2  #include <cstring>
3  #include <algorithm>
4  using namespace std;
```

```
5
6   const int maxn = 1000000 + 5;
7   const int P1 = 998244353, P2 = 1000000007;
8   const int B1 = 2, B2 = 31;
9   const int K1 = 0, K2 = 13;
10
11  typedef long long ll;
12
13  int n;
14  bool p[maxn];
15  int p1[maxn], p2[maxn];
16
17  struct H {
18      int h1, h2, l;
19      H(bool b = false) {
20          h1 = b + K1;
21          h2 = b + K2;
22          l = 1;
23      }
24      H operator + (const H &h)const {
25          H hh;
26          hh.l = l + h.l;
27          hh.h1 = (1ll * h1 * p1[h.l] + h.h1) % P1;
28          hh.h2 = (1ll * h2 * p2[h.l] + h.h2) % P2;
29          return hh;
30      }
31      bool operator == (const H &h) const {
32          return l == h.l && h1 == h.h1 && h2 == h.h2;
33      }
34      bool operator < (const H &h) const {
```

```
35            if (l != h.l)return l < h.l;
36            else if (h1 != h.h1)return h1 < h.h1;
37            elsereturn h2 < h.h2;
38        }
39    } h[maxn];
40
41    void init() {
42        memset(p, 1, sizeof(p));
43        p[0] = p[1] = false;
44        p1[0] = p2[0] = 1;
45        for (int i = 1; i <= n; i++) {
46            p1[i] = (1ll * B1 * p1[i - 1]) % P1;
47            p2[i] = (1ll * B2 * p2[i - 1]) % P2;
48            if (!p[i])continue;
49            for (int j = 2 * i; j <= n; j += i) {
50                p[j] = false;
51            }
52        }
53    }
54
55    int solve() {
56        for (int i = n; i; i--) {
57            h[i] = H(p[i]);
58            if (2 * i + 1 <= n) {
59                h[i] = h[2 * i] + h[i] + h[2 * i + 1];
60            } else if (2 * i <= n) {
61                h[i] = h[2 * i] + h[i];
62            }
63        }
64        cout << h[1].h1 << endl;
```

```
65      sort(h + 1, h + n + 1);
66      int m = unique(h + 1, h + n + 1) - (h + 1);
67      return m;
68  }
69
70  int main() {
71      cin >> n;
72      init();
73      cout << solve() << endl;
74  }
```

判断题

27. 假设程序运行前能自动将 maxn 改为 n+1，所实现的算法的时间复杂度是 O(nlogn)。（ ）

【答案】√

【解析】sort 的时间复杂度

28. 时间开销的瓶颈是 init()函数（ ）

【答案】×

【解析】init 函数的瓶颈在埃筛，O(n log log n)，所以整体的瓶颈在 sort

29.若修改常数 B1 或 K1 的值，该程序可能会输出不同呢的结果（ ）

【答案】√

【解析】有可能出现哈希冲突

单选题

30.在 solve()函数种，h[]的合并顺序可以看作是：（ ）

A.二叉树的 BFS 序

B.二叉树的先序遍历

C.二叉树的中序遍历

D.二叉树的后序遍历

【答案】C

【解析】按照字符串的合成，是先加左儿子，加自己，加右儿子，中序遍历符合条件。

31.输入“10”，输出的第一行是？（ ）

A.83

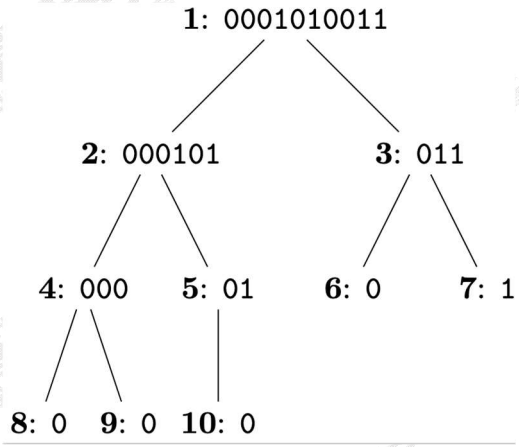
B.424

C.54

D.110101000

【答案】A

【解析】把序列构建成树的形状，依次向上合并，可以得到 H(1)对应的二进制串是 0001010011，哈希的结果为 83



32.(4 分)输入“16”，输出的第二行是？（ ）

A.7

B.9

C.10

D.12

【答案】C

【解析】画出树统计不同二进制串的数量，共 10 个

三．完善程序（单选题，每小题 3 分，共计 30 分）

（1）（序列合并）有两个长度为 N 的单调不降序列 A 和 B，序列的每个元素都是小于 10^9 的非负整数。在 A 和 B 中各取一个数相加可以得到 N^2 个和，求其中第 k 小的和。上述参数满足 N<=10^5 和 1<=K<=N^2

```
#include <iostream>
using namespace std;
const int maxn = 100005;
int n;
long long k;
int a[maxn], b[maxn];
```

```
int *upper_bound(int *a, int *an, int ai) {
    int l = 0, r = __(1)__ ;
    while (l < r) {
        int mid = (l + r) >> 1;
        if ( __(2)__ ) {
            r = mid;
        } else {
            l = mid + 1;
        }
    }
    return __(3)__ ;
}

long long get_rank(int sum) {
    long long rank = 0;
    for (int i = 0; i < n; i++) {
        rank += upper_bound(b, b + n, sum - a[i]) - b;
    }
    return rank;
}

int solve() {
    int l = 0, r = __(4)__ ;
    while (l < r) {
        int mid = ((long long)l + r) >> 1;
        if ( __(5)__ ) {
            l = mid + 1;
        } else {
            r = mid;
        }
```

```
    }
    return l;
}

int main() {
    cin >> n >> k;
    for (int i = 0; i < n; ++i)
        cin >> a[i];
    for (int i = 0; i < n; ++i)
        cin >> b[i];
    cout << solve() << endl;
    return 0;
}
```

【代码大意】根据题意，可以用样例来带入，求解。

例如：a[]={1,2,3,4,5};b[]={1,1,1,1,1}，k=4

【答案】AAAAA

33.（1）处应填（ ）

A.an-a　　B.an-a-1

C.ai　　D.ai+1

【答案】A

【解析】upper_bound 函数，找到第一个大于的位置。根据参数是指针，如果没有答案，r 应该指向最后一个没有元素位置，也就是 n，地址就是 an-a。

34.(2)处应填（ ）

A.a[mid]>ai　　B.a[mid]>=ai

C.a[mid]<ai　　D.a[mid]<=ai

【答案】A

【解析】因为是找大于的元素，所以是 A。

35.(3)处应填（ ）

A.a+l　　B.a+l+1

C.a+l-1　　D.an-l

【答案】A

【解析】返回 a 数组的第 l 个位置。

36.(4)处应填（　）

A.a[n-1]+b[n-1]　　B.a[n]+b[n]

C.2*maxn　　D.maxn

【答案】A

【解析】看主函数，执行 solve 函数。求 r 等于多少。根据范围，取不到 a[n]和 b[n]，数字是 109，答案选 A。

37.(5)处应填（　）

A.get_rank(mid)<k　　B.get_rank(mid)<=k

C.get_rank(mid)>k　　D.get_rank(mid)>=k

【答案】A

【解析】看选项，模拟运行 get_rank 函数，rank 关键字，排行，带入模拟，mid=3，小于 3 的就有 5 个了，因此，答案要小于 3。只能 AB 选项。B 选项等于的时候，L=mid+1，mid 可能是答案，B 选项排除掉了。因此选 A。

(2)（次短路）已知一个 n 个点 m 条边的有向图 G，并且给定图中的两个点 s 和 t，求次短路（长度严格大于最短路的最短路径）。如果不存在，输出一行“-1”。如果存在，输出两行，第一行表示次短路的长度，第二行表示次短路的一个方案

```
#include <cstdio>
#include <queue>
#include <utility>
#include <cstring>
using namespace std;

const int maxn = 2e5 + 10, maxm = 1e6 + 10, inf = 522133279;

int n, m, s, t;
int head[maxn], nxt[maxm], to[maxm], w[maxm], tot = 1;
int dis[maxn << 1], *dis2;
int pre[maxn << 1], *pre2;
bool vis[maxn << 1];

void add(int a, int b, int c) {
```

```
    ++tot;
    nxt[tot] = head[a];
    to[tot] = b;
    w[tot] = c;
    head[a] = tot;
}

bool upd(int a, int b, int d, priority_queue<pair<int, int> > &q) {
    if (d >= dis[b])return false;
    if (b < n)    (1)    ;
    q.push(    (2)    );
    dis[b] = d;
    pre[b] = a;
    return true;
}

void solve() {
    priority_queue<pair<int, int> >q;
    q.push(make_pair(0, s));
    memset(dis,    (3)    , sizeof(dis));
    memset(pre, -1, sizeof(pre));
    dis2 = dis + n;
    pre2 = pre + n;
    dis[s] = 0;
    while (!q.empty()) {
        int aa = q.top().second; q.pop();
        if (vis[aa])continue;
        vis[aa] = true;
        int a = aa % n;
        for (int e = head[a]; e ; e = nxt[e]) {
            int b = to[e], c = w[e];
            if (aa < n) {
                if (!upd(a, b, dis[a] + c, q))
                    (4)
            } else {
                upd(n + a, n + b, dis2[a] + c, q);
            }
        }
    }
}

void out(int a) {
    if (a != s) {
        if (a < n)  out(pre[a]);
```

```
    else out(___(5)___);
  }
  printf("%d%c", a % n + 1, " \n"[a == n + t]);
}

int main() {
  scanf("%d%d%d%d", &n, &m,&s,&t);
  s--, t--;
  for (int i = 0; i < m; ++i) {
    int a, b, c;
    scanf("%d%d%d", &a, &b, &c);
    add(a - 1, b - 1, c);
  }
  solve();
  if (dis2[t] == inf)     puts("-1");
  else {
    printf("%d\n", dis2[t]);
    out(n + t);
  }
  return 0;
}
```

38.(1)处应填（　）

A.udp(pre[b], n+b, dis[b], q)　　B.upd(a, n+b, d, q)

C.upd(pre[b], b, dis[b], q)　　D.upd(a, b, d, q)

【答案】A

【解析】

相当于 dis[0] ~ dis[n-1]存最短路，dis[n] ~ dis[2n-1]存次短路，该空之前的判断说明找到了新的最短路，那么应该将之前的最短路更新为次短路。upd(int a, int b, int d, priority_queue<pair<int, int> > &q) 用来更新从 a 更新一条到 b 的最短路为 d，当 b>n 时，相当于是 b%n 的次短路。

39.(2)处应填（　）

A.make_pair(-d, b)　　B.make_pair(d, b)

C.make_pair(b, d)　　D.make_pair(-b, d)

【答案】A

【解析】

因为 q 是用来维护距离的大根堆，所以应该存负的 dist，pair 里距离在前，节点编号在后。

40.(3)处应填（　　）

A.0xff　　B.0x1f

C.0x3f　　D.0x7f

【答案】B

【解析】

注意无解的判断是 dis2[t] == inf,而 inf = 0x1f1f1f1f,所以 memset 时应该是 0x1f

41.(4)处应填（　　）

A.upd(a, n+b, dis[a]+c, q)　　B.upd(n+a, n+b, dis2[a]+c, q)

C.upd(n+a, b, dis2[a]+c, q)　　D.upd(a, b, dis[a]+c, q)

【答案】A

【解析】

如果当前找到的路径比之前找到的最短路劣，那么我们应该再去和次短路做比较，也就是从 a 更新一条到 b 的次短路(对应节点编号 b+n)

42.(5)处应填（　　）

A.pre2[a%n]　　B.pre[a%n]

C.pre2[a]　　D.pre[a%n]+1

【答案】A

【解析】

pre2 = pre + n;所以 a 大于等于 n 时，pre2[a%n] = pre[a]